

The Notebook

May 23, 2025

0.1 # WELCOME TO THE NOTEBOOK

0.1.1 Exploratory vs Confirmatory Data Analysis

In this project, we are going to learn about two important data analysis methods **EDA** (Exploratory Data Analysis) and **CDA** (Confirmatory Data Analysis).

0.1.2 Task 1: What Is Exploratory Data Analysis (EDA)?

Definition: EDA or Exploratory Data Analysis is one of the data analysis methods where we use different statistical summaries and graphical representations to perform initial investigations on the data to discover interesting patterns, spot anomalies, and overall for a better understanding of our data.

Importing Modules

```
[1]: # Pandas Module
import pandas as pd

# Data Visualization Module
import plotly.express as px
import plotly.graph_objects as go
import plotly.offline as pyo

# Setting some default settings
pd.set_option('mode.chained_assignment', None)
pyo.init_notebook_mode()
```

Let's load our dataset

```
[2]: # encoding specified # Python engine
data = pd.read_csv('dataset.csv', engine='python', encoding='latin1')
```

Checking the data size

```
[3]: data.shape
```

```
[3]: (9994, 9)
```

Checking the data

```
[4]: data.head()
```

```
[4]:   Order Date      Customer Name      State      Category Sub-Category \
0   11/8/2016      Claire Gute      Kentucky      Furniture      Bookcases
1   11/8/2016      Claire Gute      Kentucky      Furniture      Chairs
2   6/12/2016  Darrin Van Huff  California  Office Supplies      Labels
3  10/11/2015   Sean O'Donnell      Florida      Furniture      Tables
4  10/11/2015   Sean O'Donnell      Florida  Office Supplies      Storage

                                Product Name      Sales  Quantity \
0                        Bush Somerset Collection Bookcase  261.9600      2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400      3
2  Self-Adhesive Address Labels for Typewriters b...  14.6200      2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775      5
4                        Eldon Fold 'N Roll Cart System   22.3680      2

      Profit
0   41.9136
1  219.5820
2    6.8714
3 -383.0310
4    2.5164
```

0.1.3 Task 2: EDA - Where to start?

In this task, we are going to talk about How to start our exploration.

Different column data types

How are the columns related

What are the different information in our data

Make a list of the information and start from the first

Now let's start with checking the column data types

```
[5]: data.head()
```

```
[5]:   Order Date      Customer Name      State      Category Sub-Category \
0   11/8/2016      Claire Gute      Kentucky      Furniture      Bookcases
1   11/8/2016      Claire Gute      Kentucky      Furniture      Chairs
2   6/12/2016  Darrin Van Huff  California  Office Supplies      Labels
3  10/11/2015   Sean O'Donnell      Florida      Furniture      Tables
4  10/11/2015   Sean O'Donnell      Florida  Office Supplies      Storage

                                Product Name      Sales  Quantity \
0                        Bush Somerset Collection Bookcase  261.9600      2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400      3
2  Self-Adhesive Address Labels for Typewriters b...  14.6200      2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775      5
```

```
4 Eldon Fold 'N Roll Cart System 22.3680 2
```

```
Profit
0 41.9136
1 219.5820
2 6.8714
3 -383.0310
4 2.5164
```

```
[6]: data.dtypes
```

```
[6]: Order Date      object
Customer Name      object
State              object
Category            object
Sub-Category        object
Product Name        object
Sales              float64
Quantity            int64
Profit              float64
dtype: object
```

Now let's talk about what type of information do we have in this data

0.1.4 In our data, we have the following information:

Time Information (Order Date)

Customer Information (Customer Name)

Place Information (State name)

Hierarchical Information about the products (Category, Sub-category, Product Name)

Sale Information (sales, profit, quantity)

now let's start our exploration

0.1.5 Task 3: Data Exploration: Time Information

```
[7]: data.head()
```

```
[7]: Order Date      Customer Name      State      Category Sub-Category \
0 11/8/2016      Claire Gute      Kentucky      Furniture      Bookcases
1 11/8/2016      Claire Gute      Kentucky      Furniture      Chairs
2 6/12/2016      Darrin Van Huff    California    Office Supplies  Labels
3 10/11/2015      Sean O'Donnell    Florida      Furniture      Tables
4 10/11/2015      Sean O'Donnell    Florida      Office Supplies  Storage

      Product Name      Sales      Quantity \
0      Bush Somerset Collection Bookcase  261.9600      2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400      3
```

2	Self-Adhesive Address Labels for Typewriters b...	14.6200	2
3	Bretford CR4500 Series Slim Rectangular Table	957.5775	5
4	Eldon Fold 'N Roll Cart System	22.3680	2

	Profit
0	41.9136
1	219.5820
2	6.8714
3	-383.0310
4	2.5164

What is the timespan of our data?

```
[8]: # Converting to date time
data['Order Date'] = pd.to_datetime(data['Order Date'])

# Finding the beginning time
from_ = data['Order Date'].min()

# Finding the most recent
to_ = data['Order Date'].max()
print('We have the sales information from', from_, 'to', to_)
```

We have the sales information from 2014-01-03 00:00:00 to 2017-12-30 00:00:00

Now let's sort our data by the date

```
[9]: # Sorting by date
data = data.sort_values(by = 'Order Date')
data.head()
```

```
[9]:      Order Date  Customer Name      State      Category Sub-Category \
7980 2014-01-03  Darren Powers      Texas  Office Supplies      Paper
739  2014-01-04  Phillina Ober    Illinois  Office Supplies      Labels
740  2014-01-04  Phillina Ober    Illinois  Office Supplies      Storage
741  2014-01-04  Phillina Ober    Illinois  Office Supplies      Binders
1759 2014-01-05      Mick Brown  Pennsylvania  Office Supplies      Art

      Product Name      Sales  Quantity \
7980 Message Book, Wirebound, Four 5 1/2" X 4" Form...  16.448      2
739      Avery 508  11.784      3
740      SAFCO Boltless Steel Shelving  272.736      3
741      GBC Standard Plastic Binding Systems Combs    3.540      2
1759 Avery Hi-Liter EverBold Pen Style Fluorescent ...  19.536      3

      Profit
7980  5.5512
739   4.2717
```

```

740 -64.7748
741 -5.4870
1759 4.8840

```

Some data preparation: let's extract year, month, and day from the Order Date column

```

[10]: data['Year'] = pd.DatetimeIndex(data['Order Date']).year
      data['Month'] = pd.DatetimeIndex(data['Order Date']).month
      data['Day'] = pd.DatetimeIndex(data['Order Date']).day

      data.head()

```

```

[10]:      Order Date  Customer Name      State      Category Sub-Category \
7980 2014-01-03  Darren Powers      Texas  Office Supplies      Paper
739  2014-01-04  Phillina Ober    Illinois  Office Supplies      Labels
740  2014-01-04  Phillina Ober    Illinois  Office Supplies      Storage
741  2014-01-04  Phillina Ober    Illinois  Office Supplies      Binders
1759 2014-01-05      Mick Brown  Pennsylvania  Office Supplies      Art

```

```

                                Product Name      Sales  Quantity \
7980  Message Book, Wirebound, Four 5 1/2" X 4" Form...  16.448         2
739                                Avery 508    11.784         3
740                                SAFCO Boltless Steel Shelving  272.736         3
741                                GBC Standard Plastic Binding Systems Combs    3.540         2
1759  Avery Hi-Liter EverBold Pen Style Fluorescent ...   19.536         3

```

```

      Profit  Year  Month  Day
7980  5.5512  2014     1     3
739   4.2717  2014     1     4
740 -64.7748  2014     1     4
741  -5.4870  2014     1     4
1759  4.8840  2014     1     5

```

```

[11]: # Sanity check # We can delete it
      # data['Year'].dtypes
      # data['Month'].dtypes
      # data['Day'].dtypes

```

Profit gained over time by different product categories

```

[12]: # Profit per year per category
      data_time_yearly_profit = data.groupby(['Year', 'Category']).agg({'Profit':
      ↪ 'sum'}).reset_index()
      data_time_yearly_profit.head()

```

```

[12]:      Year      Category      Profit
0  2014      Furniture  5457.7255
1  2014  Office Supplies  22593.4161

```

```

2 2014      Technology  21492.8325
3 2015      Furniture   3015.2029
4 2015 Office Supplies  25099.5338

```

Visualizing the results using a line chart

```

[13]: # Line chart
px.line(data_time_yearly_profit, x = 'Year', y = 'Profit', color = 'Category')

```

Exercise: Analyse the monthly profits gained from sales of different product categories. use a linechart to visualize your results.

```

[14]: # Profit per month per category
data_time_monthly_profit = data.groupby(['Year', 'Month', 'Category']).
    ↪agg({'Profit': 'sum'}).reset_index()
data_time_monthly_profit['Date'] = data_time_monthly_profit.Year.astype(str) +
    ↪ '-' + data_time_monthly_profit.Month.astype(str) + '-01'

# display
data_time_monthly_profit.head()

```

```

[14]:   Year  Month      Category   Profit      Date
0  2014      1      Furniture  805.4665  2014-1-01
1  2014      1 Office Supplies  788.9506  2014-1-01
2  2014      1      Technology  855.7736  2014-1-01
3  2014      2      Furniture  120.6917  2014-2-01
4  2014      2 Office Supplies  176.0910  2014-2-01

```

```

[15]: px.line(data_time_monthly_profit, x = 'Date', y = 'Profit', color = 'Category')

```

0.1.6 Data Exploration: Customer Aspect

let's see how many unique costumers do we have

```

[16]: # Number of unique in customer name
len(data['Customer Name'].unique())

```

```

[16]: 793

```

let's see the yearly change in number of unique customers

```

[17]: # Yearly change of total unique customers
customer_data = data.groupby('Year').agg({'Customer Name': 'nunique'}).
    ↪reset_index()

# Annual number of total customers
customer_data

```

```
[17]:   Year  Customer Name
      0   2014           595
      1   2015           573
      2   2016           638
      3   2017           693
```

visualizing the result

```
[18]: # Line chart of yearly total customer
      px.line(customer_data, x='Year', y='Customer Name')
```

Top 10 customers who brought the highest profit

```
[19]: # Top 10 customers
      top10_customers = data.groupby('Customer Name').agg({'Profit': 'sum'}).
      ↪reset_index().sort_values('Profit', ascending = False).head(10)
      top10_customers
```

```
[19]:   Customer Name      Profit
      730      Tamara Chand  8981.3239
      622      Raymond Buch  6976.0959
      671      Sanjit Chand  5757.4119
      334      Hunter Lopez  5622.4292
      6       Adrian Barton  5444.8055
      757      Tom Ashbrook  4703.7883
      157  Christopher Martinez  3899.8904
      431      Keith Dawkins  3038.6254
      35       Andy Reiter   2884.6208
      194      Daniel Raglin  2869.0760
```

```
[20]: px.bar(top10_customers, x = 'Customer Name', y = 'Profit')
```

0.1.7 Task 4: Data Exploration: Place (location) Aspect

Let's analyze the profits gained in different states in the US

```
[21]: geo_data = data.groupby('State').agg({'Profit': 'sum'}).reset_index()
      geo_data
```

```
[21]:   State      Profit
      0      Alabama  5786.8253
      1      Arizona -3427.9246
      2      Arkansas  4008.6871
      3      California 76381.3871
      4      Colorado -6527.8579
      5      Connecticut 3511.4918
      6      Delaware  9977.3748
      7  District of Columbia 1059.5893
```

8	Florida	-3399.3017
9	Georgia	16250.0433
10	Idaho	826.7231
11	Illinois	-12607.8870
12	Indiana	18382.9363
13	Iowa	1183.8119
14	Kansas	836.4435
15	Kentucky	11199.6966
16	Louisiana	2196.1023
17	Maine	454.4862
18	Maryland	7031.1788
19	Massachusetts	6785.5016
20	Michigan	24463.1876
21	Minnesota	10823.1874
22	Mississippi	3172.9762
23	Missouri	6436.2105
24	Montana	1833.3285
25	Nebraska	2037.0942
26	Nevada	3316.7659
27	New Hampshire	1706.5028
28	New Jersey	9772.9138
29	New Mexico	1157.1161
30	New York	74038.5486
31	North Carolina	-7490.9122
32	North Dakota	230.1497
33	Ohio	-16971.3766
34	Oklahoma	4853.9560
35	Oregon	-1190.4705
36	Pennsylvania	-15559.9603
37	Rhode Island	7285.6293
38	South Carolina	1769.0566
39	South Dakota	394.8283
40	Tennessee	-5341.6936
41	Texas	-25729.3563
42	Utah	2546.5335
43	Vermont	2244.9783
44	Virginia	18597.9504
45	Washington	33402.6517
46	West Virginia	185.9216
47	Wisconsin	8401.8004
48	Wyoming	100.1960

0.1.8 Let's create a choropleth map

Plotly uses abbreviated two-letter postal codes for state locations so it will be necessary to create a dictionary that contains conversions of the full names of states into abbreviations.


```
[22]: state_codes = {
    'Alabama': 'AL',
    'Alaska': 'AK',
    'Arizona': 'AZ',
    'Arkansas': 'AR',
    'California': 'CA',
    'Colorado': 'CO',
    'Connecticut': 'CT',
    'Delaware': 'DE',
    'District of Columbia': 'DC',
    'Florida': 'FL',
    'Georgia': 'GA',
    'Hawaii': 'HI',
    'Idaho': 'ID',
    'Illinois': 'IL',
    'Indiana': 'IN',
    'Iowa': 'IA',
    'Kansas': 'KS',
    'Kentucky': 'KY',
    'Louisiana': 'LA',
    'Maine': 'ME',
    'Maryland': 'MD',
    'Massachusetts': 'MA',
    'Michigan': 'MI',
    'Minnesota': 'MN',
    'Mississippi': 'MS',
    'Missouri': 'MO',
    'Montana': 'MT',
    'Nebraska': 'NE',
    'Nevada': 'NV',
    'New Hampshire': 'NH',
    'New Jersey': 'NJ',
    'New Mexico': 'NM',
    'New York': 'NY',
    'North Carolina': 'NC',
    'North Dakota': 'ND',
    'Ohio': 'OH',
    'Oklahoma': 'OK',
    'Oregon': 'OR',
    'Pennsylvania': 'PA',
    'Rhode Island': 'RI',
    'South Carolina': 'SC',
    'South Dakota': 'SD',
    'Tennessee': 'TN',
    'Texas': 'TX',
    'Utah': 'UT',
    'Vermont': 'VT',
```

```

    'Virginia': 'VA',
    'Washington': 'WA',
    'West Virginia': 'WV',
    'Wisconsin': 'WI',
    'Wyoming': 'WY'
}

```

let's map the abbreviated two-letter postal codes to the State column

```
[23]: geo_data.State = geo_data.State.map(state_codes)
```

```
[24]: px.choropleth(geo_data,
                    locations = 'State',
                    color = 'Profit',
                    locationmode = 'USA-states',
                    scope = 'usa',
                    title = 'Profit gained in USA states',
                    color_continuous_scale = 'Blugrn'
)
```

Exercise: Create a choropleth map to visualize the profit gained by selling technology(Category=technology) products in different states.

```
[25]: # Profit per states by Technology category
tech_data = data[data.Category == 'Technology'].groupby('State').agg({'Profit':
    ↪ 'sum'}).reset_index()

px.choropleth(tech_data,
              locations = tech_data.State.map(state_codes),
              color = 'Profit',
              locationmode = 'USA-states',
              scope = 'usa',
              title = 'Profit gained in different states based on technology_
    ↪ category',
              color_continuous_scale = 'Pbu'
)
```

NY has the highest accumulated profit from technology category

0.1.9 Task 5: Data Exploration - Hierarchical Information about the products

```
[26]: data
```

```
[26]:
```

	Order Date	Customer Name	State	Category \
7980	2014-01-03	Darren Powers	Texas	Office Supplies
739	2014-01-04	Phillina Ober	Illinois	Office Supplies
740	2014-01-04	Phillina Ober	Illinois	Office Supplies
741	2014-01-04	Phillina Ober	Illinois	Office Supplies

1759	2014-01-05	Mick Brown	Pennsylvania	Office Supplies
...	...	...	...	...
5091	2017-12-30	Jill Matthias	Colorado	Office Supplies
908	2017-12-30	Patrick O'Donnell	New York	Office Supplies
907	2017-12-30	Patrick O'Donnell	New York	Technology
1296	2017-12-30	Erica Bern	California	Office Supplies
906	2017-12-30	Patrick O'Donnell	New York	Furniture

	Sub-Category	Product Name	Sales \
7980	Paper	Message Book, Wirebound, Four 5 1/2" X 4" Form...	16.448
739	Labels	Avery 508	11.784
740	Storage	SAFCO Boltless Steel Shelving	272.736
741	Binders	GBC Standard Plastic Binding Systems Combs	3.540
1759	Art	Avery Hi-Liter EverBold Pen Style Fluorescent ...	19.536
...	...	...	...
5091	Fasteners	Bagged Rubber Bands	3.024
908	Binders	Wilson Jones Legal Size Ring Binders	52.776
907	Phones	Gear Head AU3700S Headset	90.930
1296	Binders	Cardinal Slant-D Ring Binder, Heavy Gauge Vinyl	13.904
906	Bookcases	Bush Westfield Collection Bookcases, Fully Ass...	323.136

	Quantity	Profit	Year	Month	Day
7980	2	5.5512	2014	1	3
739	3	4.2717	2014	1	4
740	3	-64.7748	2014	1	4
741	2	-5.4870	2014	1	4
1759	3	4.8840	2014	1	5
...	...	...	...	...	...
5091	3	-0.6048	2017	12	30
908	3	19.7910	2017	12	30
907	7	2.7279	2017	12	30
1296	2	4.5188	2017	12	30
906	4	12.1176	2017	12	30

[9994 rows x 12 columns]

```
[27]: # Creating new table product_data
product_data = data.groupby(['Category', 'Sub-Category']).agg({'Profit': 'sum'}).
        ↪reset_index()

# Filter, Only the profitable product (Profit > 0)
product_data = product_data[product_data.Profit > 0 ]
product_data

# Adding column
product_data['Sales'] = 'Any'
```

```
product_data
```

```
[27]:
```

	Category	Sub-Category	Profit	Sales
1	Furniture	Chairs	26590.1663	Any
2	Furniture	Furnishings	13059.1436	Any
4	Office Supplies	Appliances	18138.0054	Any
5	Office Supplies	Art	6527.7870	Any
6	Office Supplies	Binders	30221.7633	Any
7	Office Supplies	Envelopes	6964.1767	Any
8	Office Supplies	Fasteners	949.5182	Any
9	Office Supplies	Labels	5546.2540	Any
10	Office Supplies	Paper	34053.5693	Any
11	Office Supplies	Storage	21278.8264	Any
13	Technology	Accessories	41936.6357	Any
14	Technology	Copiers	55617.8249	Any
15	Technology	Machines	3384.7569	Any
16	Technology	Phones	44515.7306	Any

```
[28]: # Sun burst diagram
px.sunburst(product_data, path = ['Sales', 'Category', 'Sub-Category'], values_
↳= 'Profit')
```

Sunburst diagram shows the proportion of profit based on category and subcategory. The size of each arc is representing the amount of profit gained for each of category or subcategory.

We can click each category to navigate and see more visual representation clearly.

```
[29]: # Tree map
px.treemap(product_data, path = ['Sales', 'Category', 'Sub-Category'],
↳values='Profit')
```

The visualization by tree map is quite similar to sunburst diagram. However, it uses rectangles. The smaller rectangles represent the subcategories, the bigger ones represents the category and the biggest one is representing the ‘any’ sort of product that we have.

0.1.10 Task 6: Data Exploration: Product Sales information (Sales, Quantity, Profit)

```
[30]: data.head()
```

```
[30]:
```

	Order Date	Customer Name	State	Category	Sub-Category	\
7980	2014-01-03	Darren Powers	Texas	Office Supplies	Paper	
739	2014-01-04	Phillina Ober	Illinois	Office Supplies	Labels	
740	2014-01-04	Phillina Ober	Illinois	Office Supplies	Storage	
741	2014-01-04	Phillina Ober	Illinois	Office Supplies	Binders	
1759	2014-01-05	Mick Brown	Pennsylvania	Office Supplies	Art	

	Product Name	Sales	Quantity	\
7980	Message Book, Wirebound, Four 5 1/2" X 4" Form...	16.448	2	

739		Avery 508	11.784	3
740		SAFCO Boltless Steel Shelving	272.736	3
741		GBC Standard Plastic Binding Systems Combs	3.540	2
1759	Avery Hi-Liter EverBold Pen Style Fluorescent ...	19.536		3

	Profit	Year	Month	Day
7980	5.5512	2014	1	3
739	4.2717	2014	1	4
740	-64.7748	2014	1	4
741	-5.4870	2014	1	4
1759	4.8840	2014	1	5

Distribution Analysis on **Quantity** column

Let's check the statistical summary of the column

```
[31]: # Statistical descriptive of Quantity column
data.Quantity.describe().to_frame()
```

```
[31]:      Quantity
count  9994.000000
mean    3.789574
std     2.225110
min     1.000000
25%     2.000000
50%     3.000000
75%     5.000000
max     14.000000
```

We can notice the present of outliers based on the quick glimpse of the statistical descriptive. For instance, we have the max value of 14 that far from the upper quartile (Q75) value which is 5.

```
[32]: # Histogram of quantity
px.histogram(data, x = 'Quantity')
```

We can see the majority of quantity are between 2 and 3. Both count of these quantity are above 2400.

The distribution of this histogram is right skewed which mean the histogram contains outliers.

```
[33]: # Boxplot of Quantity
px.box(data, y = 'Quantity')
```

```
[34]: # Boxplot per year
px.box(data, y = 'Quantity', x = 'Year')
```

```
[35]: # Boxplot per category per year
px.box(data, y = 'Quantity', x = 'Category', color = 'Year')
```

Exercise: Apply distribution analysis using boxplot to the **Profit** column. using statistical summary and a box plot.

```
[36]: # Statistical descriptive of Profit column
data.Profit.describe().to_frame()
```

```
[36]:
```

	Profit
count	9994.000000
mean	28.656896
std	234.260108
min	-6599.978000
25%	1.728750
50%	8.666500
75%	29.364000
max	8399.976000

```
[37]: # Boxplot of profit
px.box(data, y = 'Profit')
```

Many outliers of the profit column

```
[38]: # Boxplot of profit per category
px.box(data, y = 'Profit', x = 'Category', color = 'Year')
```

0.1.11 Task 7: What Is Confirmatory Data Analysis (CDA)?

By definition, Confirmatory Data Analysis is the process of using statistical summary and graphical representations to evaluate the validity of an assumption about the data at hand.

We have the following assumption about our data, and we are going to use different exploration techniques we learned in the previous tasks to validate them.

Assumption 1 - Every summer technology products have the highest sale quantity compared to other

Assumption 2- In New York, there are many big companies, therefore, office supplies product has the highest sale quantity compared to other big states such as Texas, Illinois, and California

Assumption 1 - Every summer technology products have the highest sale quantity compared to other product categories.

```
[39]: # Dictionary
seasons = {
    1 : "Winter",
    2 : "Spring",
    3 : "Summer",
    4 : "Fall"
}
```

Creating **Season** column

```
[40]: # Getting index of season based on the month
data['Season'] = data.Month.astype(int)%12//3+1
data.Season = data.Season.map(seasons)
data.head()
```

```
[40]:      Order Date  Customer Name      State      Category Sub-Category \
7980 2014-01-03  Darren Powers      Texas  Office Supplies      Paper
739  2014-01-04  Phillina Ober      Illinois Office Supplies      Labels
740  2014-01-04  Phillina Ober      Illinois Office Supplies      Storage
741  2014-01-04  Phillina Ober      Illinois Office Supplies      Binders
1759 2014-01-05      Mick Brown  Pennsylvania Office Supplies      Art
```

```
      Product Name      Sales  Quantity \
7980 Message Book, Wirebound, Four 5 1/2" X 4" Form... 16.448      2
739      Avery 508      11.784      3
740      SAFCO Boltless Steel Shelving 272.736      3
741      GBC Standard Plastic Binding Systems Combs      3.540      2
1759 Avery Hi-Liter EverBold Pen Style Fluorescent ... 19.536      3
```

```
      Profit  Year  Month  Day  Season
7980   5.5512 2014      1    3  Winter
739    4.2717 2014      1    4  Winter
740  -64.7748 2014      1    4  Winter
741   -5.4870 2014      1    4  Winter
1759   4.8840 2014      1    5  Winter
```

Extracting data related to summer every year

```
[41]: # Extracting summer
summer_data = data[data.Season == 'Summer']
summer_data.head()
```

```
[41]:      Order Date  Customer Name      State      Category Sub-Category \
245 2014-06-01  Dianna Wilson  Minnesota  Office Supplies      Storage
307 2014-06-01      Corey Roper  New Jersey  Office Supplies      Art
246 2014-06-01  Dianna Wilson  Minnesota  Office Supplies      Paper
247 2014-06-01  Dianna Wilson  Minnesota  Office Supplies  Appliances
248 2014-06-01  Dianna Wilson  Minnesota  Office Supplies      Paper
```

```
      Product Name      Sales  Quantity \
245      Safco Steel Mobile File Cart 166.72      2
307 Boston Heavy-Duty Trimline Electric Pencil Sha... 289.20      6
246 Adams Telephone Message Book w/Frequently-Call... 47.88      6
247 Honeywell Enviracaire Portable HEPA Air Cleane... 1503.25      5
248      Xerox 205      25.92      4
```

```
      Profit  Year  Month  Day  Season
```

245	41.6800	2014	6	1	Summer
307	83.8680	2014	6	1	Summer
246	23.9400	2014	6	1	Summer
247	496.0725	2014	6	1	Summer
248	12.4416	2014	6	1	Summer

Aggregating data based on Year, Category, and Season columns and summing up the Quantity

```
[42]: # Aggregating data (based on year, category, season) and summing up the quantity
summer_data_agg = summer_data.groupby(['Year', 'Category', 'Season']).
    ↪agg({'Quantity': sum}).reset_index()
summer_data_agg.head()
```

```
[42]:   Year      Category Season  Quantity
0  2014      Furniture  Summer       343
1  2014  Office Supplies  Summer     1031
2  2014      Technology  Summer       306
3  2015      Furniture  Summer       386
4  2015  Office Supplies  Summer      959
```

Let's visualize our result using a grouped bar chart

```
[43]: # Creating grouped bar chart to check the validation of the assumption
px.bar(summer_data_agg, x = summer_data_agg.Year.astype('str'), y='Quantity',
    ↪color='Category', barmode='group')
```

Based on the chart above, we can see that the assumption 1 is invalide. The highest quantity of product sold was coming from the office supplies category.

Exercise: Use the analytical techniques that you've learned during the course to validate the following assumption:

Assumption 2- In New York, there are many big companies, therefore, office supplies product has the highest sale quantity compared to other big states such as Texas, Illinois

```
[44]: # data
data.head()
```

```
[44]:   Order Date  Customer Name      State  Category Sub-Category \
7980 2014-01-03  Darren Powers    Texas  Office Supplies      Paper
739  2014-01-04  Phillina Ober    Illinois  Office Supplies      Labels
740  2014-01-04  Phillina Ober    Illinois  Office Supplies      Storage
741  2014-01-04  Phillina Ober    Illinois  Office Supplies      Binders
1759 2014-01-05    Mick Brown  Pennsylvania  Office Supplies      Art

      Product Name      Sales  Quantity \
7980 Message Book, Wirebound, Four 5 1/2" X 4" Form...  16.448      2
739              Avery 508   11.784      3
740      SAFCO Boltless Steel Shelving  272.736      3
```


741	GBC Standard Plastic Binding Systems Combs	3.540	2
1759	Avery Hi-Liter EverBold Pen Style Fluorescent ...	19.536	3

	Profit	Year	Month	Day	Season
7980	5.5512	2014	1	3	Winter
739	4.2717	2014	1	4	Winter
740	-64.7748	2014	1	4	Winter
741	-5.4870	2014	1	4	Winter
1759	4.8840	2014	1	5	Winter

```
[45]: # Define 'selected' big states
big_states = ['New York', 'Texas', 'Illinois', 'California']
```

```
[46]: # Filter for Office Supplies and selected states
filtered_df = data[
    (data['Category'] == 'Office Supplies') &
    (data['State'].isin(big_states))
]

filtered_df.head()
```

```
[46]:      Order Date      Customer Name      State      Category Sub-Category \
7980 2014-01-03      Darren Powers      Texas  Office Supplies      Paper
739  2014-01-04      Phillina Ober      Illinois  Office Supplies      Labels
740  2014-01-04      Phillina Ober      Illinois  Office Supplies      Storage
741  2014-01-04      Phillina Ober      Illinois  Office Supplies      Binders
5327 2014-01-06      Lycoris Saunders  California  Office Supplies      Paper
```

	Product Name	Sales	Quantity \
7980	Message Book, Wirebound, Four 5 1/2" X 4" Form...	16.448	2
739	Avery 508	11.784	3
740	SAFCO Boltless Steel Shelving	272.736	3
741	GBC Standard Plastic Binding Systems Combs	3.540	2
5327	Xerox 225	19.440	3

	Profit	Year	Month	Day	Season
7980	5.5512	2014	1	3	Winter
739	4.2717	2014	1	4	Winter
740	-64.7748	2014	1	4	Winter
741	-5.4870	2014	1	4	Winter
5327	9.3312	2014	1	6	Winter

```
[47]: # Group by state and sum the quantity
state_quantity_comparison = (
    filtered_df
    .groupby('State', as_index=False)['Quantity']
    .sum()
```

```
.sort_values(by='Quantity', ascending=False)
)

state_quantity_comparison
```

```
[47]:
```

	State	Quantity
0	California	4566
2	New York	2585
3	Texas	2299
1	Illinois	1095

```
[48]: # Bar plot
fig = px.bar(
    state_quantity_comparison,
    x='State',
    y='Quantity',
    title='Total Quantity of Office Supplies Sold in Big States',
    labels={'Quantity': 'Total Quantity'},
    color='State',
    text='Quantity'
)

# Displaying plot
fig.show()
```

Based on our observation, the state with the highest quantity product sold is not New York. But, California. Therefore, the second assumption is not valid.

Good Luck!